

DESIGN & IMPLEMENTATION PLAN

jb_gateway_mcp

A credential-holding MCP gateway server for external API access — agents authenticate to the gateway, never to the underlying provider.

Document status	v1 — implemented, verified against a real Google account
Date	2026-08-01
Scope	MVP: Google Workspace (Gmail, Calendar, Drive)
Owner	Jithesh Bharathan

Table of Contents

1	Executive Summary
2	Goals & Non-Goals
3	Locked Requirement Decisions
4	Architecture Overview
5	Component Specifications
6	Authentication & Authorization Flows
7	Policy Model
8	Planned Tool Catalog
9	Repository Layout & Setup
10	Implementation Plan (Phased)
11	Testing Strategy
12	Security Considerations
13	Risks & Open Questions
14	Future Phases
15	Glossary

1. Executive Summary

jb_gateway_mcp is an MCP (Model Context Protocol) server that sits between AI agents/workers and external APIs (starting with Google Workspace, later read-only financial data). Its purpose is to remove credentials from the agent's reach entirely: agents call narrow, purpose-built MCP tools; the gateway holds every OAuth token and API key, decides — via a static, deny-by-default policy — whether the calling agent is permitted to use that tool, and only then talks to the external provider.

No agent, worker, or LLM ever sees a password, API key, or OAuth token. The trust boundary for v1 is the local machine: the gateway runs as a stdio MCP server, and the OS process launching it (Claude Desktop, Claude Code, or a custom local worker) is the implicit caller identity. The design keeps that identity concept abstract so a later networked version can swap in real authenticated callers (API keys/mTLS) without touching adapters or policy logic.

2. Goals & Non-Goals

Goals

- Agents authenticate to *this server only* — never directly to Google, banks, etc.
- Every external call is mediated by a deny-by-default policy check.
- Every external call is audited (who / what / when), with secrets redacted at the logging layer.
- Adding a new provider is a new adapter module, not a core rewrite.
- Token refresh and re-consent failures fail closed, never silently degrade.

Non-Goals (v1)

- No networked/multi-host transport — stdio only.
- No write/transfer capability for financial services — read-only or not integrated.
- No runtime human-approval step — authorization is static policy only.
- No multi-tenant credential isolation beyond per-account keychain entries.

3. Locked Requirement Decisions

Area	Decision	Rationale
Transport	Local stdio; core kept transport-agnostic	Fastest path to a working credential-broker; avoids building network auth before the concept is proven
MVP scope	Google APIs (Gmail, Calendar, Drive)	Well-documented OAuth2 flow, high daily utility, good adapter-pattern proving ground
Financial services	Later phase, read-only only, enforced in code not just policy	Avoids regulatory/liability surface of money movement; static-policy-only model is unsafe for writes
Credential storage	OS keychain via <code>keyring</code>	No plaintext secrets on disk; native to single-user local deployment
Authorization	Static policy file, deny-by-default, no runtime approval	Simplicity for v1; compensated for by read-only-only financial scope and audit logging

4. Architecture Overview

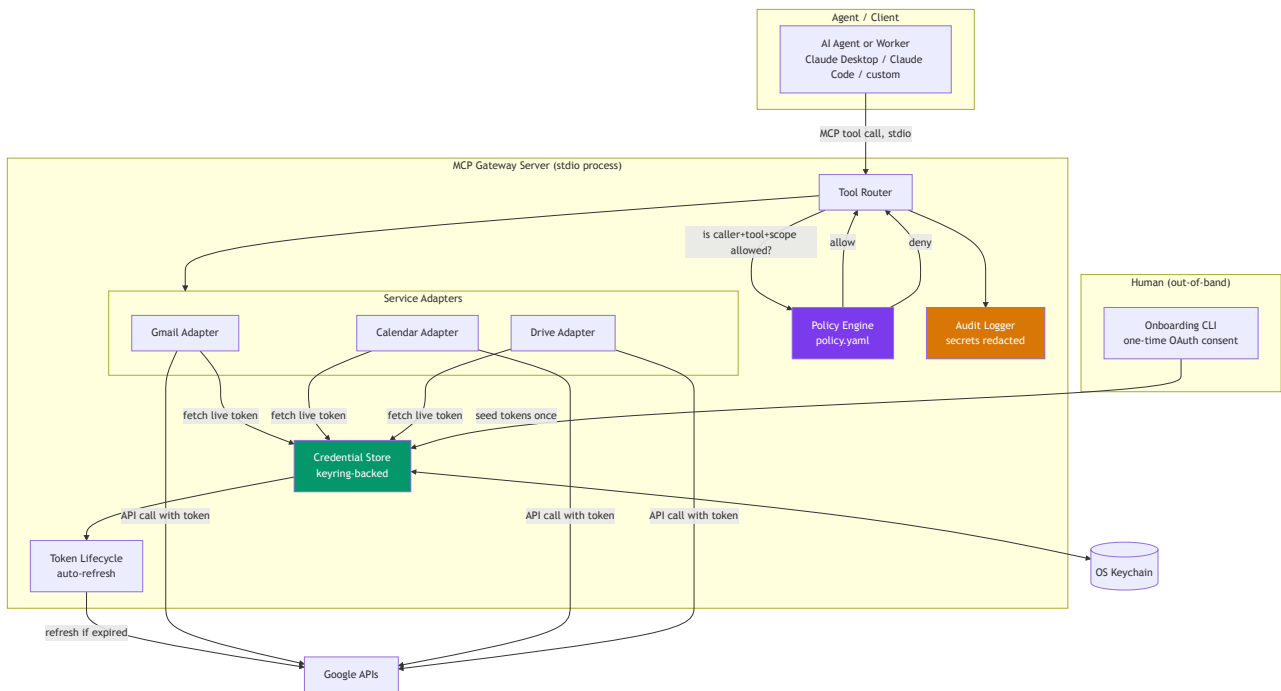


Figure 1 — Component architecture. Tokens flow only within the shaded credential loop (Credential Store ↔ Keychain ↔ Token Lifecycle ↔ Adapters) and never cross back to the Agent.

The **Tool Router** is the single entry point for every MCP tool call. It never talks to an external API directly — it consults the **Policy Engine** first, then, if allowed, delegates to the relevant **Adapter**. Adapters are the only components that both hold a live token (fetched just-in-time from the **Credential Store**) and speak to an external API. The **Audit Logger** observes every call the router makes, regardless of outcome.

5. Component Specifications

5.1 Tool Router

Dispatches an incoming MCP tool invocation to: policy check → adapter call → audit log → response.
Owns no business logic itself.

```
# illustrative – exact MCP SDK decorator/handler shape to be confirmed
# against the installed `mcp` package version at implementation time.
class ToolRouter:
    def handle(self, caller_id: str, tool_name: str, params: dict) -> ToolResult:
        decision = self.policy.check(caller_id, tool_name)
        if not decision.allowed:
            self.audit.log(caller_id, tool_name, params, outcome="denied")
            raise PermissionError(decision.reason)
        adapter = self.adapters[tool_name]
        result = adapter.invoke(params, scope=decision.scope)
        self.audit.log(caller_id, tool_name, params, outcome="success")
        return result
```

5.2 Policy Engine

Loads `policy.yaml` at startup (hot-reload optional, not required for v1). Deny-by-default: absence of an explicit grant is a denial, not an error.

```
@dataclass(frozen=True)
class PolicyDecision:
    allowed: bool
    scope: str | None
    reason: str

class PolicyEngine:
    def check(self, caller_id: str, tool_name: str) -> PolicyDecision: ...
```

5.3 Credential Store

Thin wrapper over `keyring`. Never returns a token to anything outside the adapter that requested it; never included in any object that could round-trip into a tool response or log line.

```
@dataclass(frozen=True)
class TokenRecord:
    provider: str
    account: str
    access_token: str
    refresh_token: str
```

```
expires_at: datetime
```

```
class CredentialStore:  
    def get_token(self, provider: str, account: str) -> TokenRecord: ...  
    def put_token(self, record: TokenRecord) -> None: ...
```

5.4 Token Lifecycle

Called by the Credential Store when a fetched token is expired. Refreshes via the provider's OAuth token endpoint. On `invalid_grant` (revoked refresh token), raises `NeedsReconsentError` rather than degrading to an unauthenticated call.

5.5 Audit Logger

Structured JSON Lines, one entry per tool call. Redacts by field name (`token` , `refresh_token` , `secret` , `authorization`) as a backstop independent of adapter discipline.

```
{"ts": "2026-08-01T09:12:03Z", "caller": "agent-x", "tool": "gmail.list_messages",  
  "outcome": "success", "params": {"query": "is:unread"}}
```

5.6 Adapters

One module per external service. Each adapter exposes a small, purpose-built set of MCP tools — never a raw passthrough of the underlying API. Adapters implement a shared protocol so the router/policy layer stays provider-agnostic.

```
class Adapter(Protocol):  
    tool_specs: list[ToolSpec]    # name, required scope, sensitivity  
    def invoke(self, tool_name: str, params: dict, scope: str) -> ToolResult: ...
```

Design constraint: financial adapters (later phase) subclass a `ReadOnlyAdapter` base that structurally has no write/transfer method — so a policy misconfiguration cannot grant a capability that does not exist in code.

6. Authentication & Authorization Flows

6.1 Tool call — authorized (steady state)

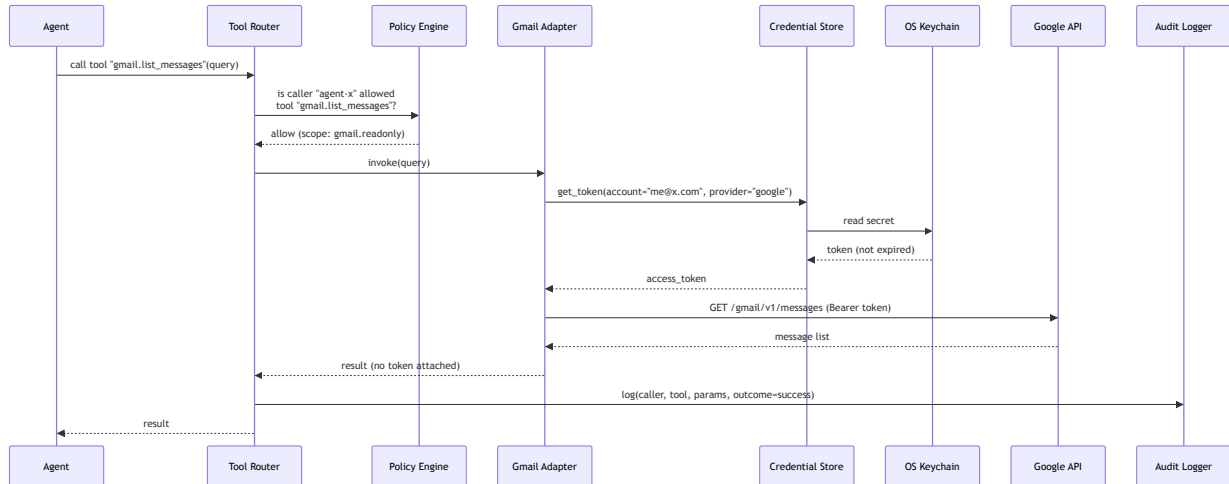


Figure 2 — A successful `gmail.list_messages` call. The token never leaves the Credential Store → Adapter boundary.

6.2 Tool call — denied by policy

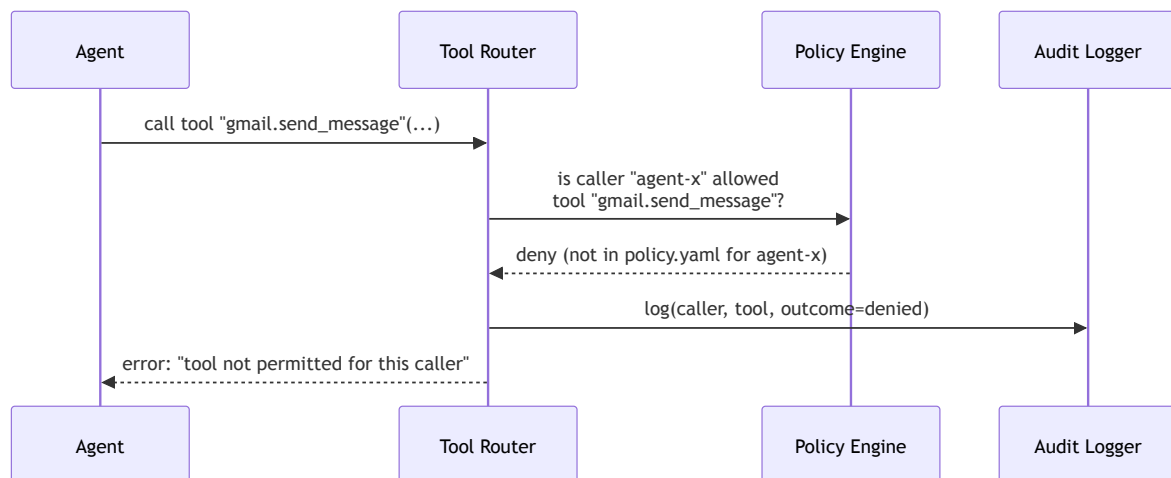


Figure 3 — No adapter is invoked on a deny; the Credential Store is never touched.

6.3 OAuth onboarding — one-time, human-driven

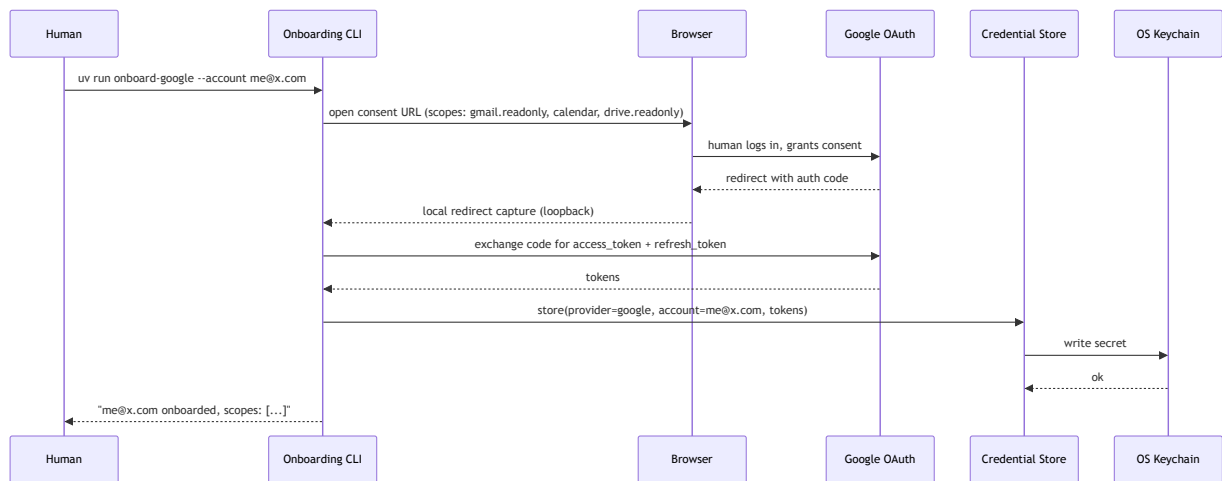


Figure 4 — A human runs a separate CLI endpoint once per account. Agents never trigger or see this flow — only its result (tools become usable).

6.4 Token refresh — automatic, inside a tool call

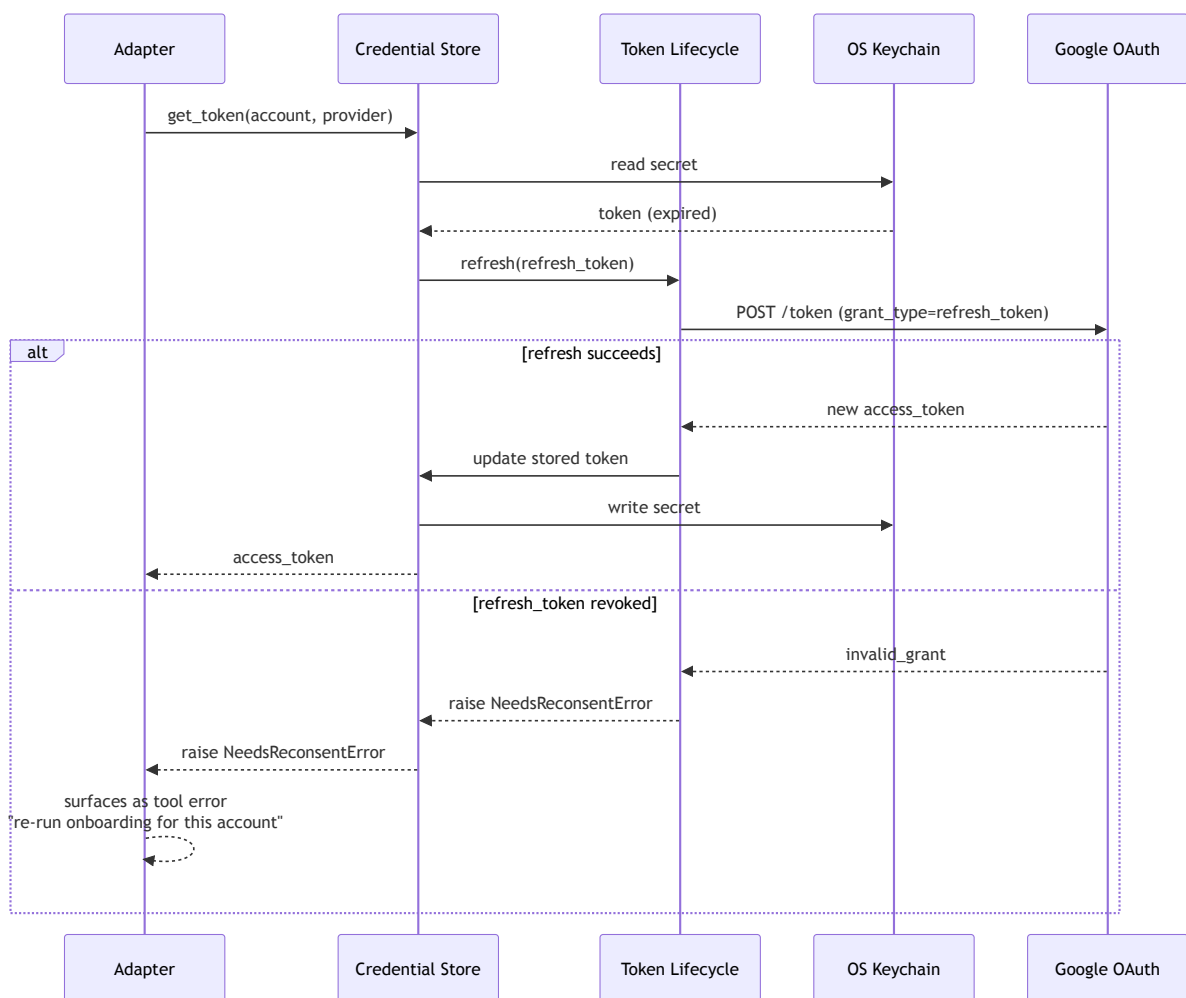


Figure 5 — Refresh failure is explicit and fails closed: a revoked refresh token surfaces as a tool error asking for re-onboarding, never a silent fallback.

7. Policy Model

`policy.yaml` — deny-by-default; each caller identity maps to explicit tool + scope grants.

```
callers:
  agent-x:
    allow:
      - tool: gmail.list_messages
        scope: gmail.readonly
      - tool: calendar.list_events
        scope: calendar.readonly
  agent-y:
    allow:
      - tool: gmail.send_message
        scope: gmail.send
```

Caller identity for stdio v1 = the local MCP client config name. No network auth is needed since the OS process boundary is the trust boundary. `router.py` / `policy.py` are written against an abstract `caller_id: str` so this becomes a real authenticated identity (API key/mTLS subject) once the networked-transport phase lands, without touching adapters.

8. Planned Tool Catalog

Tool	Adapter	Scope	Sensitivity	Notes
<code>gmail.list_messages</code>	Gmail	gmail.readonly	Low	Query/filter, no body content by default
<code>gmail.read_message</code>	Gmail	gmail.readonly	Medium	Full message body
<code>gmail.send_message</code>	Gmail	gmail.send	High	Granted per-caller only; not default
<code>calendar.list_events</code>	Calendar	calendar.readonly	Low	
<code>calendar.create_event</code>	Calendar	calendar.events	Medium	
<code>drive.list_files</code>	Drive	drive.readonly	Low	
<code>drive.read_file</code>	Drive	drive.readonly	Medium	

Exact tool set finalized at Phase 4 (see §10), gated by the policy engine before any adapter ships enabled by default.

9. Repository Layout & Setup

```
src/jb_gateway_mcp/
  server.py          # MCP server endpoint, stdio transport, tool
registration
  router.py          # dispatches tool calls -> policy check -> adapter
  policy.py           # loads policy.yaml, allow/deny decisions
  credentials.py      # keyring-backed store: get/put/refresh
  token_lifecycle.py  # refresh logic, NeedsReconsentError
  audit.py            # structured JSONL logger, redaction
  adapters/
    base.py           # shared ToolSpec + Google API client builder
    google_gmail.py   # list/read/send
    google_calendar.py # list/create events
    google_drive.py   # list/read files
  cli/
    onboard_google.py # OAuth consent flow endpoint
    uninstall.py       # revoke grant + delete keychain token, per
account
tests/
  (mirrors src/ layout)
scripts/
  start.sh            # launches the server; every client config points here
config/
  claude_desktop_config.example.json
  mcp_client_generic.example.json
.claude/skills/
  run-jb-gateway-mcp/ # project skill: spawn + drive a real MCP session to
verify the server
.mcp.json             # active Claude Code project config
policy.yaml
pyproject.toml
README.md              # step-by-step setup/usage/configuration guide (start here
to run it)
```

9.1 Setup & usage (as-built)

This summarizes what's actually running today; **README.md is the maintained, step-by-step version** — treat it as authoritative if anything here drifts.

1. **Install:** `uv sync` (Python 3.13, managed by `uv`).
2. **Create a Google OAuth client** once, in Google Cloud Console (Desktop app type, Gmail/Calendar/Drive APIs enabled) — produces a `client_secret.json` kept outside the repo.
3. **Onboard an account** (one-time, human-run, opens a browser):

```
uv run onboard-google --account you@example.com \
```

```
--client-secrets ~/.secrets/jb_gateway_mcp/client_secret.json
```

Defaults to read-only Gmail/Calendar/Drive scopes; pass `--scopes` explicitly to also grant `gmail.send` / `calendar.events`.

4. **Grant policy access** — edit `policy.yaml` (deny-by-default; nothing works until a caller has an explicit `allow` entry).
5. **Run it** via `./scripts/start.sh` — resolves the project root, checks `uv` / `policy.yaml` are present, and `exec`s into `uv run jb-gateway-mcp` so a launching client's process lifecycle reaches the real server directly. Every client config below points at this script, not at `uv` directly.
6. **Connect a client** — all use the same `JB_GATEWAY_CALLER_ID=local` (single-user, local deployment; see §7 for why one shared caller id is intentional here):
 - **Claude Code** — `.mcp.json` at the repo root, picked up automatically when the folder is opened.
 - **Claude Desktop** — copy `config/claude_desktop_config.example.json` into `claude_desktop_config.json`, with an absolute path to `scripts/start.sh`.
 - **Other MCP clients** (Cursor, Windsurf, Cline, etc.) — same `mcpServers` shape, see `config/mcp_client_generic.example.json`.
7. **Verify it's working** — the `run-jb-gateway-mcp` project skill spawns the real server via `scripts/start.sh` and drives a real MCP session against it (handshake, tool discovery, `ping`, policy-gated tool calls, audit log integrity check):

```
uv run python .claude/skills/run-jb-gateway-mcp/scripts/smoke_test.py
```

9.2 Network & ports

The gateway itself binds no port and exposes no URL — it's stdio-only, reachable only as a subprocess of whichever client launched it (matches the "Locked decisions" transport row in §1). The single exception is `onboard-google`, which briefly opens a local OAuth-redirect listener on `localhost:8080` (via `google_auth_oauthlib`'s `InstalledAppFlow.run_local_server`) for the duration of that one command only, to catch Google's OAuth redirect — nothing listens before or after it runs.

9.3 Uninstalling

Deleting the repo directory alone is not enough — stored tokens and the Google-side consent grant live outside it:

1. Remove the `jb-gateway-mcp` entry from every client config (Claude Desktop config, `.mcp.json` / `claude mcp remove` for Claude Code, etc.).
2. Run `uv run uninstall-google --account you@example.com` — revokes the grant at Google's token revocation endpoint (RFC 7009) *and* deletes the OS keychain entry in one step (prompts per

account; `--yes` to skip, `--keep-remote-grant` for local-only deletion). Falls back to local-only deletion with a manual-revoke pointer if the network call fails.

3. Delete local state outside the repo you don't want lingering: the audit log (`JB_GATEWAY_AUDIT_LOG`) and your `client_secret.json` copy.
4. Delete the project directory itself.

Full tool catalog, environment variable reference, and troubleshooting are in README.md.

10. Implementation Plan (Phased)

✓ Phase 0 — Scaffolding S

- `uv init`, `pyproject.toml` with `ruff/mypy/pytest` configured
- Add `mcp` SDK dependency; verify installed API surface against training-data assumptions before writing handlers
- Minimal `server.py` exposing one no-op tool over stdio
- **Exit criteria:** server starts, one tool callable from a local MCP client, `ruff/mypy` clean

✓ Phase 1 — Credential Store + Onboarding CLI M

- `credentials.py`: `keyring`-backed get/put
- `cli/onboard_google.py`: OAuth2 loopback consent flow, writes `TokenRecord` to keychain
- `token_lifecycle.py`: refresh logic, `NeedsReconsentError`
- **Exit criteria:** one Google account onboarded end-to-end; token readable back from keychain; simulated expiry triggers a real refresh

✓ Phase 2 — Policy Engine + Audit Log S

- `policy.py` loader + deny-by-default check
- `audit.py` JSONL writer with field-name redaction
- Wire both into `router.py` ahead of any adapter existing
- **Exit criteria:** a call to a non-existent/ungranted tool is denied and logged; no adapter is reachable without a policy grant

✓ Phase 3 — Google Adapters L

- `adapters/google_gmail.py`, `google_calendar.py`, `google_drive.py`
- Implement tool catalog from §8; parameter validation at the adapter boundary
- **Exit criteria:** each tool callable end-to-end through the full router → policy → adapter → API → audit path

✓ Phase 4 — Tests & Hardening M

- Unit tests mirroring `src/` per CLAUDE.md convention
- Refresh-failure path, policy-deny path, redaction backstop (assert no secret substrings ever appear in log output)
- Startup script, client configs (Claude Desktop/Code/generic), and a project skill for live verification — see §9.1

- **Exit criteria:** pytest green, mypy clean, manual grep of audit log across a full test run confirms zero token leakage; verified end-to-end against a real onboarded Google account

Phase 5 (Future) — Networked Transport

- HTTP/SSE transport option alongside stdio
- Real caller authentication (API keys or mTLS) replacing the process-boundary trust model

Phase 6 (Future) — Financial Adapter

- Read-only integration via an aggregator (e.g. Plaid) once selected
- Built on the `ReadOnlyAdapter` base class described in §5.6

11. Testing Strategy

- **Unit:** policy decisions (allow/deny matrix), credential store get/put against a mocked keyring backend, redaction logic against a fixed set of secret-shaped strings.
- **Integration:** full router path with a fake adapter, asserting policy is always consulted before adapter invocation (no bypass path).
- **Contract:** adapter tool specs validated against the policy schema at startup (a tool referenced in `policy.yaml` that no adapter registers should fail fast, not silently no-op).
- **Security regression:** a dedicated test asserts no `access_token` / `refresh_token` value appears in any audit log line produced during the full test suite run.

12. Security Considerations

- Tokens never appear in: tool responses, audit log entries, exceptions surfaced to the agent, or stdout/stderr.
- Audit log redacts by field name as a backstop, not just by adapter-author discipline.
- Adapters validate/sanitize all tool parameters at the boundary before building API requests — no raw parameter interpolation into URLs or queries.
- Financial adapters are architecturally restricted to read-only HTTP verbs at the base-class level, not just by policy config.
- Dependencies pinned; new dependencies flagged before adding (per project convention).
- OS keychain access itself is gated by the OS's own user-auth — no additional encryption key management needed for v1's single-user local deployment.

13. Risks & Open Questions

Risk / Question	Notes
Static policy with no runtime approval	Accepted for v1; mitigated for financial data by making writes structurally impossible rather than policy-gated. Revisit if a high-stakes write tool (e.g. <code>gmail.send_message</code>) is granted broadly.
Exact <code>mcp</code> Python SDK API surface	Not yet verified against the currently installed/latest package version — to confirm at Phase 0, not assumed from training data.
Full Gmail/Calendar/Drive operation set	Tool catalog in §8 is a starting proposal; confirm exact scopes needed before Phase 3.
Multi-agent caller identity on stdio	v1 trusts the local process; if multiple distinct agents share one machine, identity granularity is coarse until Phase 5's networked auth lands.

14. Future Phases

Beyond Phase 6, candidate directions include: additional adapters (Slack, GitHub) using the same protocol; a policy hot-reload mechanism; and, only if a genuine need for runtime human-in-the-loop approval emerges (e.g. a future write-capable financial tool), an elicitation-based confirmation step — deliberately deferred rather than built speculatively.

15. Glossary

Term	Meaning
MCP	Model Context Protocol — the interface agents use to discover and call tools
Adapter	Module translating a small set of MCP tools into calls against one external provider's API
Caller identity	The identity a policy decision is made against; process-boundary in v1, authenticated principal from Phase 5
Deny-by-default	Absence of an explicit policy grant is treated as a denial, not an oversight to warn about
Fail closed	On any ambiguous or error condition (e.g. token refresh failure), the system denies/errors rather than proceeding